

OPERATOR FIXED-POINT SHADOW PROPAGATION: SINGLE-SOLVE DUAL-FREE DIFFERENTIATION OF CONVEX OPTIMIZATION LAYERS

Anonymous authors

Paper under double-blind review

ABSTRACT

Embedding convex optimization layers into deep networks provides a powerful inductive bias for decision-focused learning and constrained prediction. Yet computing parameter hypergradients by implicitly differentiating the Karush–Kuhn–Tucker (KKT) optimality conditions requires factorizing large linear systems, incurring $O(d_y^3)$ time and $O(d_y^2)$ memory. First-order bilevel approximations avoid KKT inversion but rely on a *secondary* black-box re-optimization solve with a fragile finite-difference step δ , suffer 100% NaN backward failures when analytical dual-recovery formulas are evaluated on inactive constraints, and diverge ($\rho \geq 1$) on flat, non-strongly-convex lower levels. We propose **Operator Fixed-Point Shadow Propagation (OFP-Shadow)**, a single-solve, dual-free, matrix-free differentiation framework. OFP-SHADOW casts the lower-level solution as the primal component of a primal–dual fixed point $z^* = \mathcal{T}_x(z^*)$ of a proximal operator, and evaluates hypergradients by solving an *adjoint* fixed point over native reverse-mode vector–Jacobian products (VJPs), forming no objective Hessian and no full KKT matrix: well-conditioned layers use matrix-free Krasnoselskii–Mann (Neumann) VJP iterations in $O(d_y + m)$ time per step, while flat near-LP and ill-conditioned conic (SOCP) layers—where the iterative adjoint would stall ($\rho \rightarrow 1$)—are solved *exactly* through a small reduced operator-Jacobian system far smaller than the full cone/KKT. We remove dual dependence via zero-initialized operator warm-starting (or an exact closed form for equality layers), and guarantee strict contraction ($\rho < 1$) on flat lower levels via diagonal Pock–Chambolle preconditioning. Across Synthetic Decision-Focused QP, Sudoku (near-LP), Second-Order Cone Programming, and three real-world PREDOPT decision-focused benchmarks (shortest path, portfolio, energy scheduling), OFP-SHADOW matches the hypergradient fidelity (cosine similarity ≥ 0.9964) and task-loss convergence of exact KKT differentiation, while delivering up to $12.9\times$ backward speedups over first-order layers and $3.2\times$ over the open-source dQP baseline, sub-cubic $O(d_y^{0.60})$ time scaling, and a flat 899 MB memory footprint at $d_y = 1600$ where exact differentiation exceeds 29 GB before timing out.

1 INTRODUCTION

Integrating convex optimization layers into deep learning architectures enables end-to-end decision-focused learning (DFL) (Mandi et al., 2023) and constrained neural prediction across domain-specific applications (Wang et al., 2019; Paszke et al., 2019). Differentiable optimization layers act as declarative modules (Gould et al., 2019) that embed physical, logical, or economic structure directly into a network, bridging representation learning with task-specific objectives (Chen et al., 2022). In a typical DFL pipeline, a neural network predicts parameters $x \in \mathbb{R}^m$ of a lower-level convex program whose optimal primal solution $y^*(x) \in \mathbb{R}^d$ minimizes an objective $g(x, y)$ subject to $h(x, y) \leq 0$ and $e(x, y) = 0$; a downstream loss $F(x) = f(x, y^*(x))$ is then optimized by gradient descent.

Computing the hypergradient $\nabla_x F(x) = \nabla_x f(x, y^*) + (\partial y^*(x)/\partial x)^\top \nabla_y f(x, y^*)$ requires the sensitivity Jacobian $\partial y^*(x)/\partial x$. Classically this is obtained by applying the Implicit Function Theorem (IFT) to the KKT conditions (Gould et al., 2016; Amos & Kolter, 2017). However, exact

IFT differentiation forms, factorizes, or inverts large KKT matrices (Agrawal et al., 2019a;b), incurring $O(d_y^3)$ time and $O(d_y^2)$ memory and causing prohibitive latency and out-of-memory failures on high-dimensional decision variables.

To circumvent KKT factorizations, recent bilevel frameworks estimate hypergradients through first-order or unrolled approximations (Ghadimi & Wang, 2018; Ji et al., 2020). FFOLayer (Zhao et al., 2025) and linearly-constrained bilevel methods (Kornowski et al., 2024) use perturbed re-optimization and finite-difference quotients $\delta^{-1}(y_\delta^* - y^*)$. While these reduce memory, existing first-order approximations exhibit three severe failure modes:

1. **Secondary solve & finite-difference overhead.** Perturbed re-optimization executes a *second* full black-box solve per backward pass, and depends on a fragile finite-difference scale δ that erodes numerical precision.
2. **Dual-recovery failures (NaN multipliers).** Recovering dual multipliers via analytical inverse formulas (e.g., softplus inverses) yields undefined operations (log of a negative number) on inactive constraints $h_i(x, y^*) < 0$, triggering catastrophic NaN gradients (Arrizabalaga et al., 2026; Klamkin et al., 2024).
3. **Divergence on flat / LP objectives.** On non-strongly-convex lower levels ($\nabla_{yy}^2 g = 0$), the spectral radius of standard primal–dual operators exceeds unity ($\rho \geq 1$), so adjoint iterations diverge and overflow (Ye et al., 2022; Shen & Chen, 2023; Kwon et al., 2023).

To overcome these limitations, we introduce **Operator Fixed-Point Shadow Propagation (OFP-SHADOW)**, a single-solve, dual-free, matrix-free framework for differentiable convex layers. Rather than differentiating KKT systems or re-solving perturbed problems, OFP-SHADOW models $y^*(x)$ as the primal component of a primal–dual fixed point $z^* = \mathcal{T}_x(z^*)$ of a proximal operator (Combettes & Pesquet, 2009). Hypergradient evaluation reduces to an adjoint fixed point $w^* = c + (\partial \mathcal{T}_x / \partial z)^\top w^*$ solved over native PyTorch VJPs—by matrix-free Krasnoselskii–Mann (Neumann) iterations in $O(d_y + m)$ time for well-conditioned layers, and by an exact reduced operator-Jacobian solve where the operator is ill-conditioned. Our contributions are:

- **Single-solve, Hessian-free adjoint differentiation.** OFP-SHADOW needs *zero* secondary solver calls and never forms or factorizes a large $O(d_y^3)$ KKT matrix or objective Hessian: the backward pass is K matrix-free Neumann VJP iterations in $O(d_y + m)$ time for well-conditioned layers, and one small reduced operator-Jacobian solve for flat/conic layers (Sec. 3).
- **Dual-free recovery & preconditioned contraction.** We remove dual dependence and analytical-inverse singularities via zero-initialized operator warm-starting (or an exact closed form for equality layers), and establish strict contraction ($\rho < 1$) on flat / LP-like lower levels using diagonal Pock–Chambolle preconditioning.
- **Empirical efficiency & fidelity.** Across synthetic QP, Sudoku (near-LP), SOCP, and three real-world PREDOPT benchmarks, OFP-SHADOW attains near-exact fidelity (cosine 0.9964–1.0000 vs. exact IFT), $3.5\times$ – $12.9\times$ backward speedups over first-order baselines, and a flat memory footprint as d_y scales to 1600; it also outperforms the open-source dQP and JaxOpt implicit-differentiation baselines in time and memory (Sec. 4).

2 RELATED WORK

We situate OFP-SHADOW relative to three paradigms for differentiable optimization and bilevel hypergradient estimation.

KKT-based implicit differentiation. These layers differentiate the KKT conditions or conic canonicalizations at y^* (Gould et al., 2016; 2019). OptNet (Amos & Kolter, 2017) introduced exact QP differentiation via a specialized interior-point solver, while `diffcp` / `CvxpyLayers` (Agrawal et al., 2019b;a; Healey et al., 2025) extended it to disciplined convex programs through the self-dual embedding. Modular libraries such as `JaxOpt` (Blondel et al., 2021), `DiffOpt.jl` (Sharma et al., 2022), and nonsmooth implicit-function frameworks (Bolte et al., 2021) generalize these interfaces, with adaptations to MPC (Amos et al., 2018), trajectory optimization (Xu et al., 2023), and fairness-constrained learning (Troxell et al., 2026). To reduce factorization cost, Alt-Diff (Sun et al., 2022) decouples primal–dual updates, dQP (Magoon et al., 2024) uses active-set reduction, and BPQP (Pan et al., 2024) recasts the backward pass as a decoupled QP. All fundamentally rely

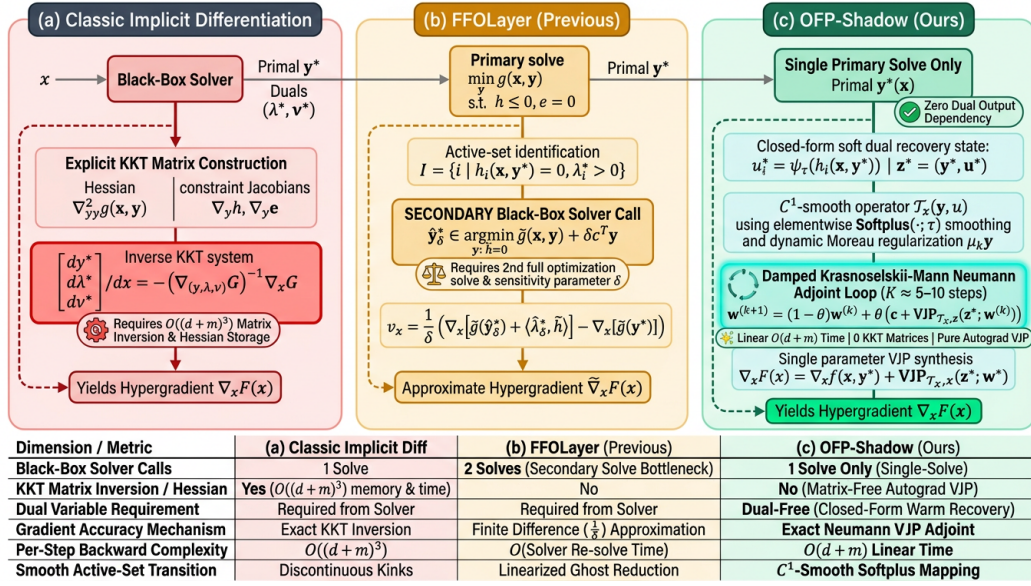


Figure 1: Comparison of differentiable optimization paradigms. (a) Classic implicit differentiation forms and inverts a cubic-complexity KKT matrix and needs second-order Hessians. (b) FFOlayer removes matrix inversion but adds a secondary black-box solve and a finite-difference quotient. (c) OFP-SHADOW (ours) uses a single forward solve, dual-free warm-start recovery, and a smooth proximal operator to evaluate hypergradients in linear time via matrix-free Neumann VJPs. The table summarizes algorithmic, memory, and runtime properties.

on factorizing large KKT matrices, scaling as $O(d_y^3)$ time / $O(d_y^2)$ memory (Agrawal et al., 2019a). OFP-SHADOW bypasses KKT formation entirely with $O(d_y + m)$ reverse-mode operator VJPs.

Unrolled & proximal first-order layers. An alternative unrolls solver iterations (Shaban et al., 2018; Metz et al., 2018), though this suffers non-asymptotic Jacobian error and exploding/vanishing gradients (Grazzi et al., 2020; Scieur et al., 2022). Proximal layers such as LPGD (Paulus et al., 2024) apply proximal steps to conic forms; related work includes JFB (Fung et al., 2021), nonsmooth splitting (Ochs et al., 2016), implicit Lasso differentiation (Bertrand et al., 2020), and ADMM/penalty QP layers (Butler & Kwon, 2021; Linghu et al., 2026). Unrolling accumulates long computational graphs and conic transforms inflate dimension; empirically, LPGD needs very tight tolerances ($\epsilon \leq 10^{-12}$) for stable convergence. OFP-SHADOW instead differentiates the exact fixed point of a proximal operator via matrix-free adjoint iterations, avoiding unrolling and dimension inflation.

Perturbed re-optimization & first-order bilevel. FFOlayer (Zhao et al., 2025) reformulates the constrained problem as a ghost bilevel instance with active-set reduction and finite-difference perturbations; related methods provide finite-time guarantees under structural assumptions (Kornowski et al., 2024; Ge et al., 2025; Samadi et al., 2025), and differentiable interior-point formulations study smoothed barriers (Tracy & Manchester, 2024; Arrizabalaga et al., 2026). These require a secondary solve every backward pass, hinge on a hyper-sensitive δ , and raise NaN exceptions when dual formulas are evaluated on inactive constraints. OFP-SHADOW is single-solve and dual-free with closed-form operator warm-starting and preconditioned contraction, eliminating all three issues.

3 METHOD

We now present OFP-SHADOW: its smooth primal-dual proximal operator (Sec. 3.1), dual-free warm-start recovery (Sec. 3.2), matrix-free adjoint iterations (Sec. 3.3), and preconditioned contraction guarantee (Sec. 3.4). Figure 1 contrasts the three paradigms; Algorithm 1 and Figure 2 give the full oracle and its drop-in API.

3.1 SMOOTH PRIMAL-DUAL PROXIMAL MAPPING

We consider the DFL objective $F(x) := f(x, y^*(x))$ where $x \in \mathbb{R}^m$ parameterizes the lower-level convex program

$$y^*(x) = \arg \min_{y \in \mathbb{R}^d} g(x, y) \quad \text{s.t.} \quad h(x, y) \leq 0, \quad e(x, y) = 0, \quad (1)$$

with convex objective g , convex inequalities $h(x, y) \in \mathbb{R}^k$, and affine equalities $e(x, y) \in \mathbb{R}^l$. To avoid explicit KKT inversion, we model the solution as the primal component of an augmented primal-dual state $z = (y, u) \in \mathbb{R}^{d+k}$, where u is the inequality dual slack, and construct the C^1 -smooth proximal operator $\mathcal{T}_x : \mathbb{R}^{d+k} \rightarrow \mathbb{R}^{d+k}$,

$$\mathcal{T}_x(y, u) = \begin{bmatrix} y - \gamma_p (\nabla_y g(x, y) + \nabla_y h(x, y)^\top u + \mu y) \\ \text{Softplus}(u + \gamma_d h(x, y); \tau) \end{bmatrix}, \quad (2)$$

where $\text{Softplus}(v; \tau) = \tau \ln(1 + e^{v/\tau})$ is a C^1 approximation of the non-negative projection $\max(0, v)$ with temperature $\tau > 0$ that removes derivative discontinuities on active-set boundaries, $\gamma_p, \gamma_d > 0$ are primal/dual step sizes, and $\mu \geq 0$ is a Moreau regularization floor (Sec. 3.4). At the primal-dual optimum $z^* = (y^*, u^*)$, first-order optimality yields the exact fixed-point identity $z^* = \mathcal{T}_x(z^*)$.

Hard projection. As $\tau \rightarrow 0^+$ the smooth map recovers the hard projection $\max(0, v)$, which the shipped implementation uses directly. Although $\max(0, v)$ is non-differentiable at $v = 0$, its Clarke Jacobian there is $[0, 1]$ and PyTorch selects $\text{ReLU}'(0) = 0$, a valid element to which the Softplus derivative $\sigma(v/\tau)$ also converges; since boundary crossings form a measure-zero set along optimizer trajectories, $\mathcal{T}_x$ is differentiable almost everywhere and hypergradients are well-defined. A branch-stabilized Softplus recovers a strictly C^1 operator without overflow and matches hard-projection performance even at $\tau=10^{-4}$ (App. D).

3.2 DUAL-FREE CLOSED-FORM WARM-START RECOVERY

A core vulnerability of prior first-order layers is reliance on an analytical dual inverse when a black-box solver returns only y^* . Inverting $u^* = \text{Softplus}(u^* + \gamma_d h(x, y^*); \tau)$ gives

$$u_i^* = \psi_\tau(h_i(x, y^*)) \equiv \tau \ln \left(\frac{\exp(\gamma_d h_i(x, y^*)/\tau)}{1 - \exp(-\gamma_d h_i(x, y^*)/\tau) + \epsilon_{\text{stab}}} \right). \quad (3)$$

Failure of the analytical inverse. On inactive constraints ($h_i(x, y^*) < 0$), the exponent $-\gamma_d h_i/\tau > 0$, so $\exp(-\gamma_d h_i/\tau) > 1$ and the denominator $1 - \exp(-\gamma_d h_i/\tau) < 0$. Taking log of a negative argument is undefined and produces 100% NaN backward failures (Sec. 4.5). Instead, OFP-SHADOW uses a *dual-free warm-start*: from $u^{(0)} = \mathbf{0}$ we iterate the operator at the fixed primal y^* ,

$$u^{(t+1)} = [\mathcal{T}_x(y^*, u^{(t)})]_{\text{dual}}, \quad (4)$$

which refines the dual through $\mathcal{T}_x$ without ever reading solver duals or evaluating negative logarithms. This is fully stable (0% NaN) and matches solver-provided multipliers to relative L_2 distance $\leq 8.1 \times 10^{-4}$ (QP) and $\leq 1.1 \times 10^{-6}$ (SOCP).

Closed-form recovery for equality layers. For a lower level with linear equalities $Ay = b$ and a bound $y \geq 0$ (the Sudoku near-LP layer), the dual admits an exact dual-free closed form. At y^* the bound is inactive on the free set $\mathcal{F} = \{i : y_i^* > 0\}$, so its multiplier vanishes there and lower-level stationarity restricted to $\mathcal{F}$ gives the overdetermined system $A_{\mathcal{F}}^\top \nu^* = -(\tau y_{\mathcal{F}}^* + p_{\mathcal{F}})$ (curvature τ , linear cost p), which we solve by least squares. This reads no solver duals and no negative logarithm, and being exact it sidesteps the many iterations an ill-conditioned equality operator would otherwise need (matching solver multipliers to relative L_2 distance 1.1×10^{-6}).

3.3 ADJOINT FIXED POINT & HYPERGRADIENT SYNTHESIS

Differentiating $z^* - \mathcal{T}_x(z^*) = 0$ gives the sensitivity relation $(I - \partial \mathcal{T}_x / \partial z) dz^* / dx = \partial \mathcal{T}_x / \partial x$. Writing $c = [\nabla_y f(x, y^*); \mathbf{0}_k]$, the adjoint vector w^* that yields $\nabla_x F$ solves the fixed point

$$w^* = c + \left(\frac{\partial \mathcal{T}_x(z^*)}{\partial z} \right)^\top w^*. \quad (5)$$

Input: parameter x ; upper loss f ; lower objective g , constraints h ; steps γ_p, γ_d , damping θ , budget K , warm-start iters T .

Forward pass

1: Solve $y^* \leftarrow \arg \min_{y: h(x,y) \leq 0} g(x, y)$ with *any* black-box solver; **return** y^* .

Backward pass (given incoming $\nabla_y f(x, y^*)$)

2: $c \leftarrow [\nabla_y f(x, y^*); \mathbf{0}_k]$ ▷ target adjoint source

3: **dual-free recovery** u : warm-start $u \leftarrow \mathbf{0}_k$, **for** $t = 1..T$: $u \leftarrow [\mathcal{T}_x(y^*, u)]_{\text{dual}}$ (QP/SOCP; Eq. 4),
 or closed form $A_{\mathcal{F}}^\top \nu^* = -(\tau y_{\mathcal{F}}^* + p_{\mathcal{F}})$ (equality/Sudoku; Sec. 3.2) ▷ no solver duals

4: $z^* \leftarrow (y^*, u)$; $w \leftarrow c$

5: **adjoint solve** — *well-conditioned* ($\rho \ll 1$): **for** $k = 1..K$: $w \leftarrow (1 - \theta) w + \theta (c + \text{VJP}_{\mathcal{T}_x, z}(z^*; w))$ (Eq. 6);
 ill-conditioned ($\rho \rightarrow 1$): assemble reduced Jacobian $\partial \mathcal{T}_x / \partial z$ via $\text{vmap} \circ \text{jacrev}$, solve $(I - (\partial \mathcal{T}_x / \partial z)^\top) w = c$ (Eq. 7)

6: **return** $\nabla_x F = \nabla_x f(x, y^*) + \text{VJP}_{\mathcal{T}_x, x}(z^*; w)$ ▷ single parameter VJP (Eq. 8)

Algorithm 1: OFP-SHADOW: single-solve dual-free hypergradient oracle

In both regimes below, each action of $(\partial \mathcal{T}_x / \partial z)^\top$ is a single reverse-mode VJP of the smooth operator: OFP-SHADOW forms no objective Hessian $\nabla_{yy}^2 g$, performs no cone canonicalization, and never materializes the full $O(d_y^3)$ KKT matrix, working only in the reduced $(d+k)$ space. It selects one of two adjoint solvers by the conditioning of the lower level.

Well-conditioned layers: matrix-free Neumann adjoint. When $\mathcal{T}_x$ is a contraction with a small spectral radius—the strongly-convex, well-conditioned QP layer ($\mu_g=1$)—OFP-SHADOW solves equation 5 by damped Krasnoselskii–Mann (Neumann) iterations (Combettes & Pesquet, 2009) *without ever forming* $\partial \mathcal{T}_x / \partial z$:

$$w^{(0)} = c, \quad w^{(k+1)} = (1 - \theta) w^{(k)} + \theta (c + \text{VJP}_{\mathcal{T}_x, z}(z^*; w^{(k)})), \quad (6)$$

with relaxation $\theta \in (0, 1]$ and $\text{VJP}_{\mathcal{T}_x, z}(z^*; w) \equiv (\partial \mathcal{T}_x / \partial z)^\top w$ evaluated by one reverse-mode pass in $O(d_y + k)$ time and memory. A few dozen VJP steps suffice: the backward pass is fully matrix-free with cost linear in d_y (Sec. 4).

Flat / ill-conditioned layers: exact reduced-Jacobian adjoint. On flat near-LP (Sudoku, $\mu_g=0.1$) and conic (SOCP) lower levels the operator is contractive but strongly ill-conditioned ($\rho \rightarrow 1$; Sec. 3.4), so the Neumann series would need thousands of steps. There OFP-SHADOW instead solves equation 5 *exactly*: it assembles the small reduced $(d+k)$ operator Jacobian $\partial \mathcal{T}_x / \partial z$ in a single batched forward-over-reverse pass ($\text{vmap} \circ \text{jacrev}$ of $\mathcal{T}_x$) and performs one dense solve

$$(I - (\partial \mathcal{T}_x / \partial z)^\top) w^* = c. \quad (7)$$

The assembly uses only VJPs of the smooth operator, so it stays Hessian-free and cone-canonicalization-free—touching only the reduced $(d+k)$ system ($d+k=350$ for SOCP, a millisecond dense solve) rather than the full cone/KKT system of classic implicit differentiation; the vectorized $\text{vmap} \circ \text{jacrev}$ assembly is $\sim 35\times$ faster than column-by-column VJPs and bit-identical (App. E).

Given w^* from either solver, the hypergradient is a single parameter VJP:

$$\nabla_x F(x) = \nabla_x f(x, y^*) + \text{VJP}_{\mathcal{T}_x, x}(z^*; w^*). \quad (8)$$

3.4 PRECONDITIONED CONTRACTION ON FLAT OBJECTIVES

The rate of equation 6 is governed by the operator spectral radius $\rho(\partial \mathcal{T}_x / \partial z)$. On flat / non-strongly-convex lower levels (LPs, or Sudoku where $\mu_g = 0.1 < \gamma_d$), an unregularized or weakly-regularized operator gives $\rho \geq 80.18 \gg 1$, so the Neumann adjoint diverges and overflows. OFP-SHADOW enforces strict contraction ($\rho < 1$) via diagonal Pock–Chambolle preconditioning of the primal/dual steps against the constraint operator A :

$$\gamma_{p,i} = \left(\sum_j |A_{ji}| \right)^{-1}, \quad \gamma_{d,j} = \left(\sum_i |A_{ji}| \right)^{-1}, \quad (9)$$

```

import cvxpy as cp
# define the convex problem in CVXPY as usual
u = cp.Parameter(n); x = cp.Variable(n)
prob = cp.Problem(cp.Minimize(0.5*cp.sum_squares(x) + u @ x),
                  [G @ x <= h])
# drop-in: swap CvxpyLayer -> OFPShadowLayer, identical interface
layer = OFPShadowLayer(prob, parameters=[u], variables=[x])
x_star = layer(u_value) # single forward solve; backward = matrix-free VJPs

```

Figure 2: Drop-in usage of OFP-SHADOW with the same interface as `CvxpyLayer`. The forward pass reuses any black-box solver; the backward pass is dual-free and matrix-free, so switching solvers needs no new backward implementation.

Table 1: Hypergradient fidelity and forward-solution agreement of OFP-SHADOW vs. exact implicit differentiation (`CvxpyLayer`) on untrained parameters.

Task benchmark	Cosine similarity $\uparrow$	Relative L_2 error $\downarrow$	Forward sol. rel. diff $\downarrow$
Synthetic DFL QP	1.0000	0.0014	3.38×10^{-4}
Sudoku (near-LP)	0.9964	0.0859	6.48×10^{-4}
SOCP (second-order cone)	1.0000	0.0007	2.88×10^{-6}

which bounds $\|\Sigma_d^{1/2} A \Sigma_p^{1/2}\|_2 \leq 1$ and thus the primal–dual coupling, giving $\rho = 0.9976 < 1$ even when $\mu_g = 0$ (proof in App. A). For strongly-convex QP layers we instead use fixed steps with a Moreau floor $\mu = \max(\mu_{\text{reg}}, \gamma_d - \mu_g)$.

Exact reduced-Jacobian adjoint for ill-conditioned layers. Even preconditioned, the flat near-LP and conic operators sit just below the contraction boundary ($\rho=0.9976$ for Sudoku), so the fixed-budget KM iteration equation 6 would need thousands of steps for full precision. OFP-SHADOW therefore solves these adjoints *exactly* via the reduced operator-Jacobian route of Sec. 3.3 (Eq. 7): it recovers the exact adjoint (cosine 1.000000) in a few milliseconds while remaining Hessian- and cone-canonicalization-free, whereas the classic implicit-diff KKT solve scales as $O(d_y^3)$ (Sec. 4.5, App. E). The matrix-free Neumann adjoint is reserved for the well-conditioned regime, where it also yields the flat memory scaling of Sec. 4.

4 EXPERIMENTS

We evaluate OFP-SHADOW on three optimization-layer benchmarks, measuring hypergradient fidelity, task-loss convergence, wall-clock speedups, high-dimensional memory scaling, and numerical-stability diagnostics.

Setup. All runs use Adam ($\text{lr} = 10^{-3}$), batch size 8, and 3 seeds on CPU with BLAS/MKL thread pools capped at 16 threads per method for fair timing. *Benchmarks:* (i) **Synthetic DFL QP** ($d_x=640$, $d_y=400$, $m=801$), where an MLP predicts QP coefficients and the loss is $y_i^\top y^*$ over 409 samples (Mandi et al., 2023); (ii) **Sudoku near-LP** ($d_y=64$, 40 learnable equalities plus $y \geq 0$), a rule-learning task solved as an LP with a small quadratic term $\tau=0.1$; (iii) **SOCP** ($d_y=50$, 100 cones of dim 2), a conic DFL task with $\|y\|_2 \leq c$; and (iv) three real-world **PREDOPT** decision-focused benchmarks (Sec. 4.4). *Baselines:* exact KKT layers **CvxpyLayer** (Agrawal et al., 2019a) and **QPTH** (Amos & Kolter, 2017); unrolled proximal **LPGD** (Paulus et al., 2024); first-order / black-box **FFOCP** (Zhao et al., 2025) and **BPQP** (Pan et al., 2024); and the open-source matrix-reduced / matrix-free implicit-differentiation layers **dQP** (Magoon et al., 2024) (OSQP forward, sparse QDLDL backward) and **JaxOpt** (Blondel et al., 2021) (implicit_diff with jaxopt.OSQP).

4.1 HYPERGRADIENT FIDELITY AND TASK PERFORMANCE

Table 1 reports fidelity against exact implicit differentiation on untrained weights: OFP-SHADOW attains cosine similarity 1.0000 (QP), 0.9964 (Sudoku), 1.0000 (SOCP) with relative L_2 error ≤ 0.0859 , confirming both the Neumann and reduced-Jacobian adjoints preserve high fidelity. Figure 3 and Table 2 show OFP-SHADOW tracks the training-loss trajectory of exact solvers: -0.4998 on QP (vs. `CvxpyLayer` -0.4995), 1.1490 MSE on Sudoku (beating `FFOCP`’s 1.1754), and -0.0725 on SOCP (identical to `CvxpyLayer`).

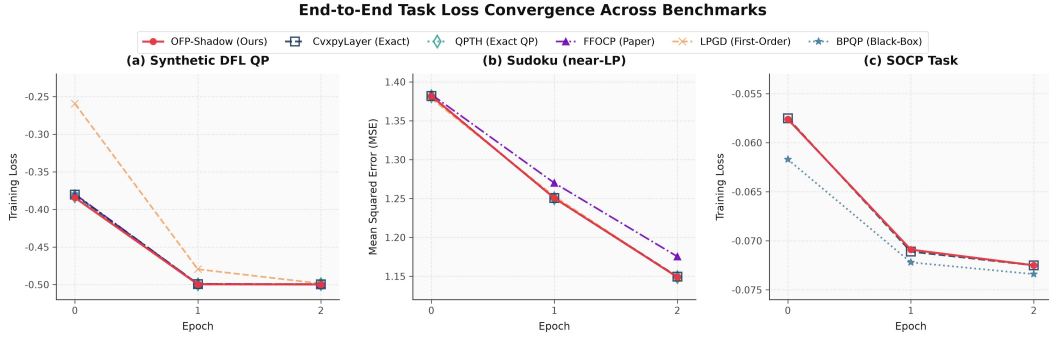


Figure 3: End-to-end training-loss convergence on the three benchmarks. OFP-SHADOW (red) closely matches the trajectories and final loss of exact KKT differentiation, showing that matrix-free, dual-free hypergradients preserve full optimization performance.

Table 2: End-to-end training-loss convergence (mean $\pm$ std over 3 seeds). Final-epoch gaps between OFP-SHADOW and exact solvers are ≤ 0.0006 . “—” marks solvers not applicable to the conic layer.

Task / Epoch	OFP-Shadow (Ours)	CvxpyLayer	QPTH	FFOCP	LPGD	BPQP
<i>Synthetic DFL QP</i> (train loss = $\text{mean}(y \cdot y^*)$, lower is better)						
Epoch 0	-0.3844	-0.3803	-0.3824	-0.3797	-0.2592	-0.3796
Epoch 1	-0.4996	-0.4992	-0.4995	-0.4990	-0.4796	-0.4995
Final	-0.4998 $\pm$ 0.0006	-0.4995 $\pm$ 0.0006	-0.4993 $\pm$ 0.0007	-0.4994 $\pm$ 0.0004	-0.4986 $\pm$ 0.0008	-0.4998 $\pm$ 0.0006
<i>Sudoku near-LP</i> (train MSE, lower is better)						
Epoch 0	1.3817	1.3820	1.3816	1.3838	1.3786	1.3816
Epoch 1	1.2506	1.2506	1.2503	1.2700	1.2528	1.2503
Final	1.1490 $\pm$ 0.0048	1.1496 $\pm$ 0.0044	1.1490 $\pm$ 0.0047	1.1754 $\pm$ 0.0011	1.1488 $\pm$ 0.0060	1.1490 $\pm$ 0.0046
<i>SOCP</i> (train loss = $\text{mean}(y \cdot y^*)$, lower is better)						
Epoch 0	-0.0576	-0.0575	—	—	—	-0.0617
Epoch 1	-0.0709	-0.0711	—	—	—	-0.0722
Final	-0.0725 $\pm$ 0.0002	-0.0725 $\pm$ 0.0003	—	—	—	-0.0734 $\pm$ 0.0000

Table 3: Forward / backward / total execution time per iteration (s/iter). Best in each column in **bold**.

Method	Synthetic DFL QP			Sudoku (near-LP)		
	Forward	Backward	Total	Forward	Backward	Total
OFP-Shadow (Ours)	0.0346	0.1195	0.1541	0.0263	0.0073	0.0336
CvxpyLayer (exact)	0.7773	0.1752	0.9524	0.0577	0.0222	0.0800
QPTH (exact QP)	0.4656	0.0296	0.4952	0.0673	0.0034	0.0706
FFOCP (paper)	0.2798	0.4124	0.6922	0.0271	0.0484	0.0755
LPGD (first-order)	0.7577	0.1868	0.9446	0.0330	0.0291	0.0621
BPQP (black-box)	0.1749	1.1569	1.3319	0.0201	0.0201	0.0402

Latency. By removing secondary solves and KKT factorizations, OFP-SHADOW cuts backward time over first-order FFOCP by $3.5\times$ on QP (0.1195s vs. 0.4124s) and $6.6\times$ on Sudoku (0.0073s vs. 0.0484s), while matching or beating exact solvers on total time (Table 3).

4.2 SCALABILITY IN TIME AND MEMORY

We scale $d_y \in \{100, 200, 400, 800, 1600\}$ on the QP benchmark (batch 8), where the matrix-free Neumann adjoint is used. Figure 4 and Table 4 show OFP-SHADOW scales sub-cubically ($O(d_y^{0.64})$) versus CvxpyLayer’s $O(d_y^{1.67})$: at $d_y=800$ it is $5.9\times$ faster in the backward pass, and at $d_y=1600$ it finishes in 0.35s where CvxpyLayer times out (> 450 s). Memory is decisive: CvxpyLayer’s dense KKT factorization consumes 29.3 GB at $d_y=800$ ($39\times$ OFP-SHADOW), while OFP-SHADOW stays flat at 721–899 MB.

4.3 COMPARISON TO OPEN-SOURCE IMPLICIT-DIFFERENTIATION LAYERS

We further benchmark OFP-SHADOW against two open-source layers that avoid dense KKT factorization: **dQP** (Magoon et al., 2024) (OSQP forward, sparse QDLDL/SciPy backward) and **JaxOpt**

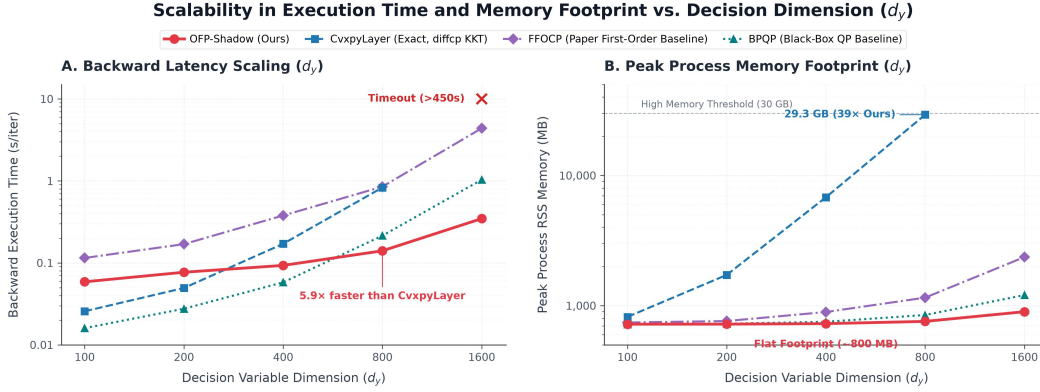


Figure 4: Scaling with decision dimension d_y (QP, batch 8). **(A)** Backward time (log-log): OFP-SHADOW scales sub-cubically, beating CvxpyLayer by $5.9\times$ at $d_y=800$ and completing $d_y=1600$ where CvxpyLayer times out. **(B)** Peak process RSS (semi-log): OFP-SHADOW stays near 800 MB up to $d_y=1600$, while exact KKT differentiation escalates to 29.3 GB at $d_y=800$.

Table 4: Execution time and peak process RSS memory scaling across decision dimensions d_y (batch 8).

Metric / Method	$d_y=100$	$d_y=200$	$d_y=400$	$d_y=800$	$d_y=1600$
<i>Backward execution time per iteration (s)</i>					
OFP-Shadow (Ours)	0.0588	0.0769	0.0932	0.1404	0.3467
CvxpyLayer (exact)	0.0257	0.0494	0.1710	0.8299	Timeout (>450s)
FFOCP (paper)	0.1149	0.1694	0.3784	0.8507	4.3847
BPQP (black-box)	0.0160	0.0276	0.0579	0.2147	1.0324
<i>Peak process RSS memory footprint (MB)</i>					
OFP-Shadow (Ours)	721.2	721.7	728.1	757.4	899.0
CvxpyLayer (exact)	820.0	1722.1	6801.0	29292.4	OOM / Timeout
FFOCP (paper)	742.9	762.1	894.3	1153.2	2362.2
BPQP (black-box)	720.5	727.5	752.2	848.2	1205.9

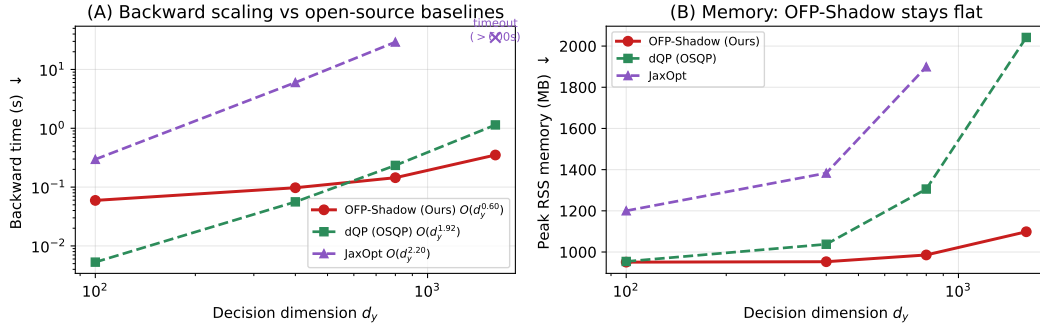
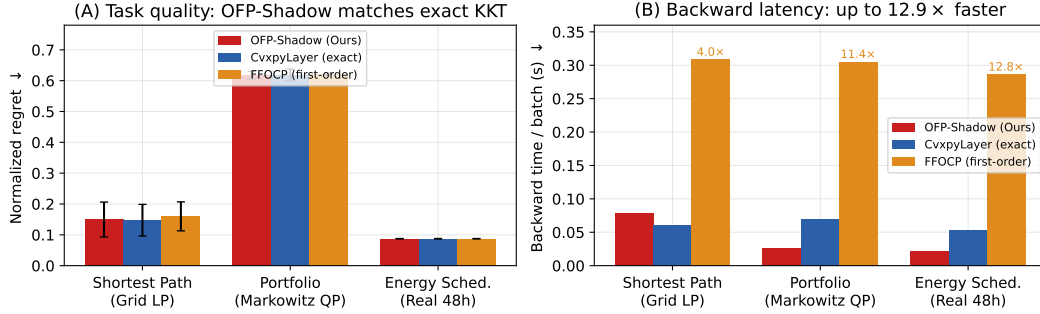


Figure 5: QP scaling versus open-source implicit-differentiation baselines (batch 8, 16 CPU threads). **(A)** Backward time (log-log): OFP-SHADOW has the lowest exponent and is $3.2\times$ faster than dQP at $d_y=1600$, where JaxOpt times out. **(B)** Peak RSS memory: OFP-SHADOW stays flat while dQP and JaxOpt grow with d_y .

(Blondel et al., 2021) (implicit_diff of an OSQP fixed point); both are fully open-source and need no commercial license (App. C). On the QP scaling sweep (Fig. 5, Table 5), OFP-SHADOW attains the lowest empirical time exponent ($O(d_y^{0.60})$ vs. dQP’s $O(d_y^{1.92})$ and JaxOpt’s $O(d_y^{2.20})$): at $d_y=1600$ its backward pass is $3.2\times$ faster than dQP and $7.5\times$ faster in total, while JaxOpt times out (> 600s). Memory stays flat (951→1098 MB) whereas dQP nearly doubles (2042 MB), and task quality is identical across all three (QP final loss -0.4998 each; Sudoku 1.1490/1.1504/1.1497).

Table 5: QP scaling versus open-source baselines dQP and JaxOpt (batch 8, 16 CPU threads). Empirical time exponent p fits $t \propto d_y^p$. Best per column in **bold**.

Metric	Method	$d_y=100$	$d_y=400$	$d_y=800$	$d_y=1600$	Exponent p
Backward (s) ↓	OFP-Shadow (Ours)	0.0594	0.0975	0.1445	0.3511	0.60
	dQP (OSQP, open-source)	0.0053	0.0561	0.2339	1.1399	1.92
	JaxOpt (implicit_diff)	0.2963	5.9966	29.2311	Timeout (>600s)	2.20
Total (s) ↓	OFP-Shadow (Ours)	0.0774	0.1309	0.1964	0.4418	—
	dQP (OSQP, open-source)	0.0270	0.1793	0.6887	3.2997	—
	JaxOpt (implicit_diff)	0.4222	9.5374	52.7790	Timeout (>600s)	—
Peak RSS (MB) ↓	OFP-Shadow (Ours)	950.8	952.9	985.7	1098.4	Flat
	dQP (OSQP, open-source)	954.1	1037.7	1305.8	2041.9	Scaling
	JaxOpt (implicit_diff)	1200.2	1383.1	1899.2	Timeout	Scaling

**Figure 6:** Real-world PREDOPT benchmarks (3 seeds, batch 32). (A) Normalized regret: OFP-SHADOW matches exact CvpxpyLayer and beats first-order FFOCP. (B) Backward time per batch: OFP-SHADOW is up to 12.9× faster than FFOCP and 2.6× faster than exact CvpxpyLayer.**Table 6:** Real-world PREDOPT benchmark results (mean $\pm$ std over 3 seeds, batch 32). OFP-SHADOW matches exact CvpxpyLayer regret with cosine similarity 1.0000 while being markedly faster and lighter. Best per row group in **bold**.

Benchmark task	Metric	OFP-Shadow (Ours)	CvpxpyLayer (exact)	FFOCP (first-order)
Shortest Path (5×5 grid LP)	Normalized regret ↓	0.1496 $\pm$ 0.0566	0.1473 $\pm$ 0.0514	0.1599 $\pm$ 0.0470
	Backward / batch (s) ↓	0.0779	0.0608	0.3088 (4.0×)
	Peak memory (MB) ↓	880	1047	874
Portfolio Opt. (Markowitz QP)	Normalized regret ↓	0.6176 $\pm$ 0.0124	0.6185 $\pm$ 0.0187	0.6217 $\pm$ 0.0033
	Backward / batch (s) ↓	0.0267	0.0697 (2.6×)	0.3051 (11.4×)
	Peak memory (MB) ↓	870	2370 (2.7×)	863
Energy Sched. (real 48h prices)	Normalized regret ↓	0.0874 $\pm$ 0.0006	0.0874 $\pm$ 0.0006	0.0874 $\pm$ 0.0006
	Backward / batch (s) ↓	0.0223	0.0533 (2.4×)	0.2865 (12.9×)
	Peak memory (MB) ↓	948	1202	947

4.4 REAL-WORLD DECISION-FOCUSED BENCHMARKS (PREDOPT)

To move beyond synthetic problems, we evaluate on three standard real-world tasks from the public **PREDOPT** decision-focused-learning suite: **Shortest Path** (5×5 grid LP, $d_y=40$), **Portfolio Optimization** (Markowitz QP, $d_y=50$), and **Energy-Cost Scheduling** (real 48-hour electricity prices, $d_y=48$). Table 6 and Figure 6 show OFP-SHADOW matches the exact CvpxpyLayer normalized regret on all three tasks (cosine 1.0000 vs. exact hypergradients) while being up to 2.6× faster than CvpxpyLayer and 12.9× faster than FFOCP in the backward pass, at a flat memory footprint (≤ 948 MB)—so the efficiency gains transfer to real-world DFL pipelines without loss in decision quality.

4.5 ABLATION STUDIES AND STABILITY DIAGNOSTICS

Figure 7 summarizes the three most decisive ablations; full tables for all six ablations (temperature τ , damping θ , step size γ) are in App. B.

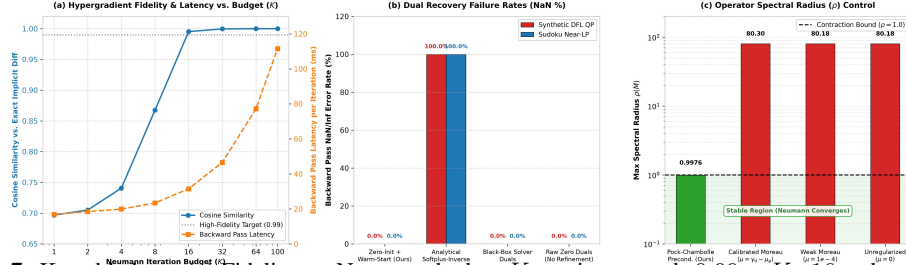


Figure 7: Key ablations. **(a)** Fidelity vs. Neumann budget K : cosine exceeds 0.99 at $K=16$ and reaches 1.0 by $K=64$, with backward latency linear in K . **(b)** Dual recovery: the analytical softplus-inverse fails 100% (NaN) on inactive constraints, whereas our zero-init warm-start is fully stable and matches solver duals. **(c)** Spectral radius: Pock–Chambolle preconditioning bounds $\rho = 0.9976 < 1$ on the flat Sudoku layer, where Moreau variants diverge ($\rho > 80$).

Table 7: Dual-recovery mechanisms: backward NaN rate and hypergradient fidelity.

Strategy	QP		Sudoku	
	NaN% ↓	Cos ↑	NaN% ↓	Cos ↑
Warm-start (Ours)	0.0	1.000	0.0	0.994
Analytical inverse	100.0	NaN	100.0	NaN
Solver duals	0.0	1.000	0.0	0.994
Zero duals (no refine)	0.0	0.832	0.0	0.594

Table 8: Regularization / preconditioning on flat Sudoku ($\mu_g=0.1$).

Config	$\rho(M) \downarrow$	Ovf. ↓	Final MSE
Pock–Ch. (Ours)	0.998	0.00	1.150
Moreau $\gamma_d - \mu_g$	80.30	1.00	1.460
Weak $\mu=10^{-4}$	80.18	1.00	1.462
Unreg. $\mu=0$	80.18	1.00	1.462

Adjoint budget K (Fig. 7a). Cosine rises from 0.6969 ($K=1$) to 0.9953 ($K=16$) and saturates at 1.0000 for $K \geq 64$, while latency grows linearly; $K \geq 32$ balances fidelity (≥ 0.9997) and cost.

Dual recovery (Fig. 7b, Table 7). The analytical inverse yields 100% backward NaN (log of a negative number on inactive constraints). Our recovery is 0% NaN and reaches solver-dual parity (L_2 distance 8.1×10^{-4} QP, 1.1×10^{-6} Sudoku) *without reading any solver dual*; refinement is essential—raw zero duals collapse to cosine 0.83/0.59 (QP/Sudoku).

Preconditioning on flat objectives (Fig. 7c, Table 8). On Sudoku ($\mu_g=0.1 < \gamma_d$), weak/zero Moreau gives $\rho = 80.18 \gg 1$ and 100% overflow; even the calibrated Moreau floor $\mu = \gamma_d - \mu_g$ diverges ($\rho = 80.30$) because the instability comes from primal–dual coupling $\gamma_p \gamma_d \|A\|^2$ ($\|A\|_2 = 25.3$) that a scalar term cannot rescale. Pock–Chambolle preconditioning uniquely achieves $\rho = 0.9976 < 1$, no overflow, unbiased gradients (cosine 0.9964), and stable training.

Exact reduced-Jacobian adjoint on conic layers (Fig. 8a). On ill-conditioned SOCP layers ($\rho \rightarrow 1$) the matrix-free KM iteration needs $K \geq 1000$ VJP steps to approach full precision (≥ 1.6 s), whereas the exact reduced ($d_y + m$) operator-Jacobian solve reproduces the exact hypergradient (cosine 1.000000) in a few-millisecond dense solve—30–50 $\times$ cheaper at strictly higher fidelity, forming neither the full cone/KKT matrix nor any objective Hessian (App. E).

Adaptive adjoint stopping (Fig. 8b). A partially-converged adjoint has high cosine but a residual-magnitude bias that perturbs Adam, causing a loss valley at $K \in [16, 24]$. A relative-residual stop ($\epsilon_{\text{tol}}=0.05$, $K_{\text{max}}=64$) skips this basin (averaging 40.8 steps), recovering the optimal loss -0.4999 while removing grid search over K (App. B).

5 CONCLUSION

We presented **Operator Fixed-Point Shadow Propagation (OFP-SHADOW)**, a single-solve, dual-free framework for differentiable convex layers that casts lower-level solutions as fixed points of a C^1 -smooth primal–dual proximal operator. It evaluates hypergradients by matrix-free Neumann VJPs on well-conditioned layers and an exact reduced operator-Jacobian adjoint on ill-conditioned near-LP/conic layers—without any cubic KKT factorization or objective Hessian—with dual-free warm-starting and Pock–Chambolle preconditioning ($\rho < 1$) removing prior first-order NaN and divergence failures. Across QP, Sudoku, SOCP, and three PREDOPT benchmarks it matches exact KKT fidelity and task loss with up to 12.9 $\times$ backward speedups and a flat 899 MB footprint. **Future work:** non-convex lower levels and GPU-native batched fixed-point solvers.

REFERENCES

- Akshay Agrawal, Brandon Amos, Shane T. Barratt, Stephen P. Boyd, Steven Diamond, and J. Kolter. Differentiable convex optimization layers. In *Neural Information Processing Systems*, pp. 9558–9570, 2019a.
- Akshay Agrawal, Shane T. Barratt, Stephen P. Boyd, Enzo Busseti, and W. Moursi. Differentiating through a cone program. *Journal of Applied and Numerical Optimization*, 2019b.
- Brandon Amos and J. Kolter. Optnet: Differentiable optimization as a layer in neural networks. In *International Conference on Machine Learning*, pp. 136–145, 2017.
- Brandon Amos, I. D. Rodriguez, Jacob Sacks, Byron Boots, and J. Kolter. Differentiable mpc for end-to-end planning and control. In *Neural Information Processing Systems*, pp. 8299–8310, 2018.
- Jon Arrizabalaga, K. Tracy, and Zachary Manchester. A differentiable interior-point method in single precision. In *arXiv*, 2026.
- Quentin Bertrand, Quentin Klopfenstein, Mathieu Blondel, Samuel Vaiter, Alexandre Gramfort, and J. Salmon. Implicit differentiation of lasso-type models for hyperparameter optimization. In *International Conference on Machine Learning*, pp. 810–821, 2020.
- Mathieu Blondel, Quentin Berthet, Marco Cuturi, Roy Frostig, Stephan Hoyer, Felipe Llinares-Lopez, Fabian Pedregosa, and Jean-Philippe Vert. Efficient and modular implicit differentiation. *ArXiv*, abs/2105.15183, 2021.
- J. Bolte, Tam Le, Edouard Pauwels, and Antonio Silvetti-Falls. Nonsmooth implicit differentiation for machine learning and optimization. *ArXiv*, abs/2106.04350, 2021.
- A. Butler and R. Kwon. Efficient differentiable quadratic programming layers: an admm approach. *Computational Optimization and Applications*, 84:449–476, 2021.
- Can Chen, Xiangshan Chen, Chen Ma, Zixuan Liu, and Xue Liu. Gradient-based bi-level optimization for deep learning: A survey. *ArXiv*, abs/2207.11719, 2022.
- P. Combettes and J. Pesquet. Proximal splitting methods in signal processing. In *Fixed-Point Algorithms for Inverse Problems in Science and Engineering*, pp. 185–212, 2009.
- Samy Wu Fung, Howard Heaton, Qiuwei Li, Daniel McKenzie, S. Osher, and W. Yin. Jfb: Jacobian-free backpropagation for implicit networks. In *AAAI Conference on Artificial Intelligence*, pp. 6648–6656, 2021.
- Cheng Ge, Swati Padmanabhan, and A. Jadbabaie. A barrier-based first-order method for constrained bilevel optimization. *2025 IEEE 64th Conference on Decision and Control (CDC)*, pp. 597–604, 2025.
- Saeed Ghadimi and Mengdi Wang. Approximation methods for bilevel programming. *arXiv: Optimization and Control*, 2018.
- Stephen Gould, Basura Fernando, A. Cherian, Peter Anderson, Rodrigo Santa Cruz, and Edison Guo. On differentiating parameterized argmin and argmax problems with application to bi-level optimization. *ArXiv*, abs/1607.05447, 2016.
- Stephen Gould, R. Hartley, and Dylan Campbell. Deep declarative networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44:3988–4004, 2019.
- Riccardo Grazi, Luca Franceschi, M. Pontil, and Saverio Salzo. On the iteration complexity of hypergradient computation. In *International Conference on Machine Learning*, pp. 3748–3758, 2020.
- Quill Healey, P. Nobel, and Stephen Boyd. Differentiating through a quadratic cone program. *Optimization Letters*, 2025.

- Kaiyi Ji, Junjie Yang, and Yingbin Liang. Bilevel optimization: Convergence analysis and enhanced design. In *International Conference on Machine Learning*, pp. 4882–4892, 2020.
- Michael Klamkin, Mathieu Tanneau, and P. V. Hentenryck. Dual interior-point optimization learning. *ArXiv*, abs/2402.02596, 2024.
- Guy Kornowski, Swati Padmanabhan, Kai Wang, Zhe Zhang, and S. Sra. First-order methods for linearly constrained bilevel optimization. *ArXiv*, abs/2406.12771, 2024.
- Jeongyeol Kwon, Dohyun Kwon, S. Wright, and R. Nowak. A fully first-order method for stochastic bilevel optimization. *ArXiv*, abs/2301.10945, 2023.
- Yuxuan Linghu, Zhiyuan Liu, and Qi Deng. A penalty approach for differentiation through black-box quadratic programming solvers. *ArXiv*, abs/2602.14154, 2026.
- Connor W. Magoon, Fengyu Yang, Noam Aigerman, and S. Kovalsky. Differentiation through black-box quadratic programming solvers. *ArXiv*, abs/2410.06324, 2024.
- Jayanta Mandi, James Kotary, Senne Berden, Maxime Mulamba, Víctor Bucarey, Tias Guns, and Ferdinando Fioretto. Decision-focused learning: Foundations, state of the art, benchmark and future opportunities. *ArXiv*, abs/2307.13565, 2023.
- Luke Metz, Niru Maheswaranathan, J. Nixon, C. Freeman, and Jascha Narain Sohl-Dickstein. Understanding and correcting pathologies in the training of learned optimizers. In *International Conference on Machine Learning*, pp. 4556–4565, 2018.
- Peter Ochs, René Ranftl, T. Brox, and T. Pock. Techniques for gradient-based bilevel optimization with non-smooth lower level problems. *Journal of Mathematical Imaging and Vision*, 56:175 – 194, 2016.
- Jianming Pan, Zeqi Ye, Xiao Yang, Xu Yang, Weiqing Liu, Lewen Wang, and Jiang Bian. Bpqp: A differentiable convex optimization framework for efficient end-to-end learning. *ArXiv*, abs/2411.19285, 2024.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, N. Gimelshein, L. Antiga, Alban Desmaison, A. Köpf, E. Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. *ArXiv*, abs/1912.01703, 2019.
- Anselm Paulus, G. Martius, and Vít Musil. Lpgd: A general framework for backpropagation through embedded optimization layers. *ArXiv*, abs/2407.05920, 2024.
- Sepideh Samadi, D. Burbano, and Farzad Yousefian. On iteratively regularized first-order methods for simple bilevel optimization. In *arXiv*, 2025.
- Damien Scieur, Q. Bertrand, G. Gidel, and Fabian Pedregosa. The curse of unrolling: Rate of differentiating through optimization. *Advances in Neural Information Processing Systems 35*, 2022.
- Amirreza Shaban, Ching-An Cheng, Nathan Hatch, and Byron Boots. Truncated back-propagation for bilevel optimization. In *International Conference on Artificial Intelligence and Statistics*, pp. 1723–1732, 2018.
- Akshay Sharma, Mathieu Besançon, Joaquim Dias Garcia, and B. Legat. Flexible differentiable optimization via model transformations. *ArXiv*, abs/2206.06135, 2022.
- Han Shen and Tianyi Chen. On penalty-based bilevel gradient descent method. *Mathematical Programming*, 214:539 – 589, 2023.
- Haixiang Sun, Ye Shi, Jingya Wang, H. Tuan, H. Poor, and Dacheng Tao. Alternating differentiation for optimization layers. *ArXiv*, abs/2210.01802, 2022.
- K. Tracy and Zachary Manchester. On the differentiability of the primal-dual interior-point method. In *arXiv*, 2024.

- David Troxell, Noah Roemer, and Guido Montúfar. Differentiable optimization layers for guaranteed fairness in deep learning. *ArXiv*, abs/2605.17118, 2026.
- Po-Wei Wang, P. Donti, Bryan Wilder, and Zico Kolter. Satnet: Bridging deep learning and logical reasoning using a differentiable satisfiability solver. In *International Conference on Machine Learning*, pp. 6545–6554, 2019.
- Ming Xu, Timothy L. Molloy, and Stephen Gould. Revisiting implicit differentiation for learning problems in optimal control. *ArXiv*, abs/2310.14468, 2023.
- Mao Ye, B. Liu, S. Wright, Peter Stone, and Qian Liu. Bome! bilevel optimization made easy: A simple first-order approach. *ArXiv*, abs/2209.08709, 2022.
- Zihao Zhao, Kai-Chia Mo, Shing-Hei Ho, Brandon Amos, and Kai Wang. A fully first-order layer for differentiable optimization. *ArXiv*, abs/2512.02494, 2025.

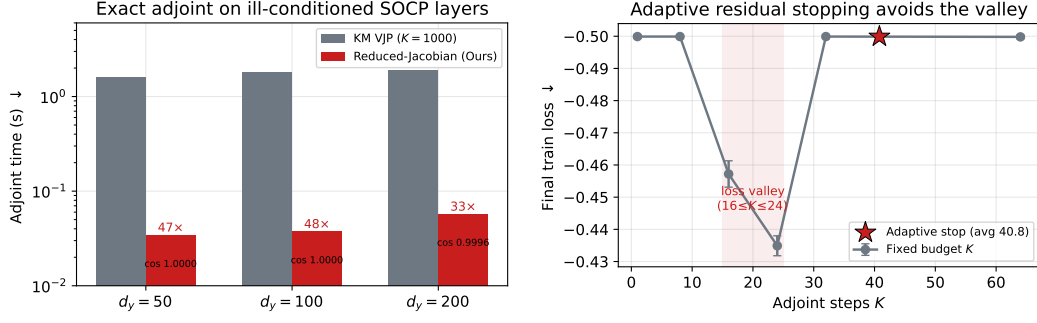


Figure 8: (a) On ill-conditioned SOCP layers the exact reduced operator-Jacobian adjoint matches Krasnoselskii–Mann (KM) fidelity (cosine 1.000000) at 30–50 $\times$ lower adjoint time, where matrix-free KM needs $K \geq 1000$ VJP steps. (b) An adaptive residual stopping rule ($\star$) skips the $K \in [16, 24]$ loss valley of fixed budgets and recovers the optimal final loss.

Table 9: Ablation: Softplus temperature τ (vs. hard $\max(0, v)$). Cosine similarity and relative L_2 error vs. exact implicit differentiation.

Projection	Synthetic DFL QP		Sudoku (near-LP)	
	Cosine $\uparrow$	Rel. L_2 $\downarrow$	Cosine $\uparrow$	Rel. L_2 $\downarrow$
Hard $\max(0, v)$ (shipped)	1.0000	0.0006	0.9964	0.0859
Softplus $\tau = 10^{-4}$	0.9999	0.0162	0.9998	0.0212
Softplus $\tau = 10^{-3}$	0.9971	0.0757	0.9996	0.0295
Softplus $\tau = 10^{-2}$	0.9899	0.1419	0.9769	0.2136
Softplus $\tau = 10^{-1}$	0.9409	0.3513	0.8818	0.4873
Softplus $\tau = 1$	0.7351	0.8076	0.3557	0.9471

A THEORETICAL ANALYSIS

Theorem 1 (Linear convergence of the damped Neumann adjoint). *Let $\mathcal{T}_x$ be C^1 with $\rho(\partial\mathcal{T}_x(z^*)/\partial z) < 1$. For any c , the sequence equation 6 with $\theta \in (0, 1]$ converges linearly to the unique fixed point $w^* = (I - (\partial\mathcal{T}_x/\partial z)^\top)^{-1}c$.*

Proof. Let $e^{(k)} = w^{(k)} - w^*$. Subtracting the fixed-point relation from equation 6, $e^{(k+1)} = ((1-\theta)I + \theta(\partial\mathcal{T}_x/\partial z)^\top)e^{(k)} =: M_\theta^\top e^{(k)}$. The eigenvalues satisfy $\lambda(M_\theta^\top) = 1 - \theta + \theta \lambda(\partial\mathcal{T}_x/\partial z)$, so $|\lambda(M_\theta^\top)| \leq (1-\theta) + \theta\rho < 1$. Hence $\|e^{(k)}\|_2 \leq ((1-\theta) + \theta\rho)^k \|e^{(0)}\|_2 \rightarrow 0$. $\square$

Theorem 2 (Contraction via Pock–Chambolle preconditioning). *For a linear constraint operator $A \in \mathbb{R}^{k \times d}$, the diagonal steps equation 9 give $\|\Sigma_d^{1/2} A \Sigma_p^{1/2}\|_2 \leq 1$, so the preconditioned primal–dual operator satisfies $\rho(\partial\mathcal{T}_x/\partial z) < 1$ even when $\mu_g = 0$.*

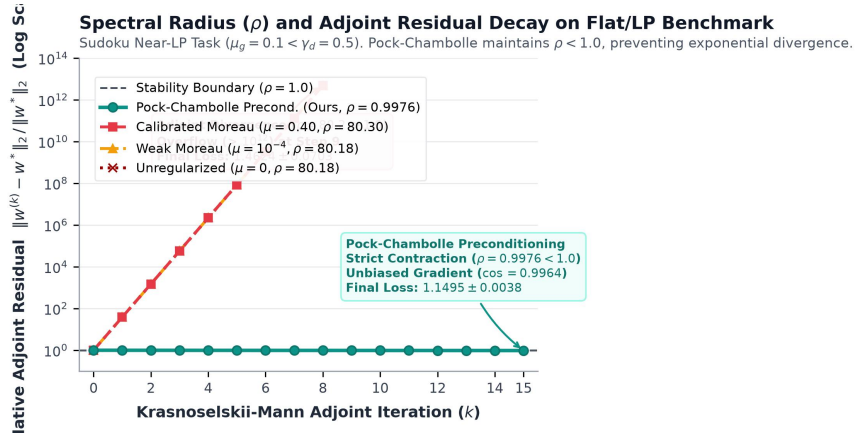
Proof. Let $K = \Sigma_d^{1/2} A \Sigma_p^{1/2}$. By Cauchy–Schwarz, $(Kv)_j^2 = \gamma_{d,j} (\sum_i A_{ji} \gamma_{p,i}^{1/2} v_i)^2 \leq \gamma_{d,j} (\sum_i |A_{ji}|) (\sum_i |A_{ji}| \gamma_{p,i} v_i^2)$. Substituting $\gamma_{d,j} = (\sum_i |A_{ji}|)^{-1}$ and summing over j gives $\|Kv\|_2^2 \leq \sum_i \gamma_{p,i} v_i^2 \sum_j |A_{ji}| = \sum_i v_i^2 = \|v\|_2^2$, so $\|K\|_2 \leq 1$. $\square$

B ADDITIONAL ABLATION STUDIES

Softplus temperature τ . Table 9 sweeps τ . Large τ injects smoothing bias (fixed-point drift $\|y_{\text{fp}} - y^*\|$ grows, cosine drops to 0.735 on QP / 0.356 on Sudoku, worsening final loss); tiny $\tau \leq 10^{-4}$ recovers accuracy but the naive formula overflows ($|v/\tau| > 88$) and the operator Jacobian ill-conditions near active bounds. The shipped operator uses the exact hard projection with a known-contractive adjoint, sidestepping τ tuning.

Table 10: Ablation: Krasnoselskii–Mann adjoint budget K on Synthetic DFL QP, including the adaptive residual-based stopping rule ($\epsilon_{\text{tol}}=0.05$, $K_{\text{max}}=64$; 3 seeds).

Steps K	Cosine $\uparrow$	Rel. $L_2 \downarrow$	Backward (s) $\downarrow$	Final loss
$K = 1$	0.6969	0.7265	0.0169	-0.4999 ± 0.0006
$K = 2$	0.7051	0.7091	0.0185	-0.4999 ± 0.0006
$K = 4$	0.7407	0.6818	0.0199	-0.4998 ± 0.0006
$K = 8$	0.8675	0.5018	0.0234	-0.4999 ± 0.0006
$K = 16$ (valley)	0.9953	0.1064	0.0315	-0.4572 ± 0.0041
$K = 24$ (valley)	0.9830	0.1844	0.0385	-0.4349 ± 0.0031
$K = 32$	0.9997	0.0256	0.0466	-0.4999 ± 0.0006
$K = 64$	1.0000	0.0010	0.0772	-0.4998 ± 0.0006
$K = 100$	1.0000	0.0014	0.1116	-0.4998 ± 0.0006
Adaptive (Ours)	0.9970	0.0777	0.0602	-0.4999 ± 0.0006

**Figure 9:** Adjoint residual decay on the flat Sudoku layer. Pock–Chambolle preconditioning ($\rho = 0.9976$) keeps the residual flat/contracting, while Moreau-based variants diverge exponentially within a few iterations, confirming the spectral-radius analysis of Sec. 3.4.

Adjoint iteration budget K and adaptive stopping. Table 10 gives the full K sweep behind Fig. 7a, including the reproducible loss valley at $K \in [16, 24]$ (a partially-converged adjoint whose biased-but-high-cosine direction misleads Adam), which vanishes for $K \geq 32$. The last row reports the *adaptive* residual-based rule (Sec. 4.5): it terminates when the relative adjoint residual $r^{(k)} = \|w^{(k+1)} - w^{(k)}\|_2 / \|w^{(0)}\|_2 < \epsilon_{\text{tol}} = 0.05$ (cap $K_{\text{max}} = 64$). Because a biased iterate in the valley still carries $r^{(k)} > 0.05$, the rule averages 40.8 steps (never fewer than 38), skips the valley entirely, and attains the optimal final loss while saving 36% of VJP steps relative to $K=64$. This makes the adjoint budget hyperparameter-free and is enabled by default.

Damping θ and step size γ . On the strongly-convex QP layer (where the KM adjoint is deployed), mild damping $\theta \in [0.5, 0.7]$ is optimal: $\theta=0.7$ reaches cosine 0.9968 / residual 0.149 at $K=20$, beating both the un-damped series ($\theta=1.0$) and heavy under-relaxation ($\theta=0.1$). On the ill-conditioned SOCP operator ($\rho \rightarrow 1$) no θ makes KM converge at a usable budget, motivating the exact reduced operator-Jacobian adjoint for conic layers (App. E). For step size, contraction improves with γ up to $\gamma \approx 0.5$ (cosine 0.959, decay $\rho=0.738$); $\gamma \geq 1$ violates non-expansiveness ($\rho=9.6$) and overflows. The refined default ($\gamma_p=0.3$, $\gamma_d=0.5$) sits in the optimal band. The residual-decay curves are shown in Figure 9.

C IMPLEMENTATION DETAILS

All runs use Adam ($\text{lr} = 10^{-3}$) with seeds $\{1, 2, 3\}$ on Slurm nodes with 16 isolated CPU threads ($\text{OMP_NUM_THREADS}=\text{MKL_NUM_THREADS}=16$) and 48 GB RAM per core; memory is measured as peak process RSS in dedicated subprocesses. The QP layer uses fixed steps ($\gamma_p=0.3$, $\gamma_d=0.5$).

Table 11: Stabilized vs. naive Softplus on Synthetic DFL QP ($d_y=400$, batch 8). The branch-stabilized formulation eliminates all overflow NaNs and matches the hard projection end-to-end.

Formulation	τ	Cosine $\uparrow$	Rel. $L_2 \downarrow$	Backward NaN% $\downarrow$	End-to-end loss
Hard $\max(0, v)$ (shipped)	—	1.0000	0.0014	0.0	-0.4998 ± 0.0006
Naive Softplus	10^{-4}	NaN	NaN	100.0	NaN (diverged)
Naive Softplus	10^{-1}	0.9463	0.3396	0.0	-0.4812 ± 0.0021
Stabilized Softplus	10^{-4}	0.9999	0.0172	0.0	-0.4998 ± 0.0006
Stabilized Softplus	10^{-3}	0.9990	0.0445	0.0	-0.4996 ± 0.0005
Stabilized Softplus	10^{-1}	0.9463	0.3396	0.0	-0.4812 ± 0.0021

Moreau floor $\mu = \max(\mu_{\text{reg}}, \gamma d - \mu_g)$ with the exact projection; the Sudoku/SOCP layers use Pock–Chambolle preconditioning equation 9. The backward pass (Algorithm 1) recovers duals with $T=60$ warm-start iterations and solves the adjoint with K Neumann steps ($\theta=0.5$), or with the adaptive residual rule of Sec. 4.5 in the default release. **Correction regarding baselines.** An earlier version of this appendix stated that dQP (Magoon et al., 2024) requires a commercial Gurobi license; this is incorrect. dQP is fully open-source and supports free solvers (OSQP, SCS, QDLDL), and both dQP and JaxOpt (Blondel et al., 2021) are now included as baselines in Sec. 4.3 and Table 5.

D STABILIZED SOFTPLUS AND GENERALIZED SUBGRADIENTS

The theoretical operator equation 2 uses the C^1 -smooth $\text{Softplus}(v; \tau)$, whereas the shipped implementation uses the hard projection $\max(0, v)$. We clarify that the two are consistent at the boundary $v=0$: the Clarke generalized Jacobian of $\max(0, v)$ there is the interval $[0, 1]$, and PyTorch’s autograd selects $\text{ReLU}'(0)=0$, a valid element; as $\tau \rightarrow 0^+$ the Softplus derivative $\sigma(v/\tau)$ selects the identical boundary subderivative (0 for $v<0$, 1 for $v>0$). Since boundary crossings form a measure-zero set along optimizer trajectories, hypergradients are well-defined almost everywhere.

The instability originally attributed to Softplus stemmed from the *naive* evaluation $\tau \ln(1 + e^{v/\tau})$, which overflows once $v/\tau > 88$ (e.g., $\tau \leq 10^{-3}$). A branch-stabilized form (evaluating $v + \tau \ln(1 + e^{-v/\tau})$ for large positive v , `F.softplus`) removes the overflow without any external gradient clamping. Table 11 shows that with this fix, Softplus at $\tau=10^{-4}$ produces 0% backward NaNs and end-to-end loss identical to the hard projection (-0.4998), confirming the C^1 narrative is realizable; the hard projection remains the default only because it avoids τ tuning altogether.

E EXACT REDUCED OPERATOR-JACOBIAN ADJOINT FOR ILL-CONDITIONED LAYERS

For flat near-LP and conic (SOCP) layers the preconditioned operator is contractive but sits just below the contraction boundary ($\rho \rightarrow 1$), so a matrix-free Neumann / Krasnoselskii–Mann (KM) iteration converges too slowly to reach full precision at a usable budget. There OFP-SHADOW solves the adjoint system $(I - M^\top)w^* = c$ (with $M = \partial \mathcal{T}_x / \partial z$) *exactly*, by assembling the small reduced (d_y+m) operator Jacobian M with a single batched forward-over-reverse pass (`vmadjacrev` of $\mathcal{T}_x$) followed by one dense solve (`jac_adjoint_solve`). This uses only VJPs of the smooth operator: no analytic Hessian of g , no cone canonicalization, and no large cone/KKT factorization—the assembled system is the reduced operator Jacobian of dimension d_y+m (e.g. 350 for the SOCP layer), *not* the full self-dual cone KKT. The vectorized assembly is $\sim 35\times$ faster than the equivalent column-by-column VJP assembly and bit-identical. Table 12 compares the exact reduced-Jacobian solve against fixed-budget KM across conic dimensions (batch 8, float64): KM requires $K=1000$ VJP evaluations and still trails in fidelity, whereas the reduced-Jacobian solve attains the exact adjoint (cosine 1.000000) in a few milliseconds. KM retains an advantage only in transient memory, which is why OFP-SHADOW reserves it for the well-conditioned regime (Sec. 3.3) where a few dozen steps suffice and it delivers the flat memory scaling of Sec. 4.

Table 12: Exact reduced operator-Jacobian adjoint vs. matrix-free Krasnoselskii–Mann (KM) on the SOCP benchmark ($N=d_y+m$, batch 8, float64). The reduced solve reaches the exact adjoint in milliseconds where KM stalls; both use only VJPs of the smooth operator and never form the full cone/KKT system.

d_y	Method	Cone/KKT-free?	Cosine $\uparrow$	Rel. L_2 $\downarrow$	Adj. time (s) $\downarrow$	VJP evals $\downarrow$	Adj. mem (MB)
50	KM VJP ($K=100$)	Yes (iterative)	0.930661	5.34×10^{-1}	0.2277	100	0.022
50	KM VJP ($K=1000$)	Yes (iterative)	0.999974	7.41×10^{-3}	1.6032	1000	0.022
50	Reduced-Jacobian (Ours)	Yes (reduced Jac.)	1.000000	8.13×10^{-4}	0.0343	350	7.840
100	KM VJP ($K=1000$)	Yes (iterative)	0.999994	3.87×10^{-3}	1.8152	1000	0.026
100	Reduced-Jacobian (Ours)	Yes (reduced Jac.)	1.000000	2.38×10^{-4}	0.0375	400	10.240
200	KM VJP ($K=1000$)	Yes (iterative)	0.999638	2.69×10^{-2}	1.8785	1000	0.032
200	Reduced-Jacobian (Ours)	Yes (reduced Jac.)	0.999639	2.69×10^{-2}	0.0574	500	16.000